Basic — a Streamlit UI — Solution

> Back to the exercise · Hint 1 · Hint 2

Story: this is the fastest possible way to get a real web UI — a plain Python script that Streamlit turns into a live, interactive page with no HTML or JavaScript written at all, plus a chat version that remembers the conversation using `st.session_state`. **If not:** every project would stay a terminal script that nobody but you could ever run or share.

The pipeline (shared by every exercise)

```
# practice/pipeline.py — in a real project this calls your agent/RAG pipeline
def classify(text: str) -> str:
    lowered = text.lower()
    for word in ["love", "great", "amazing", "good"]:
        if word in lowered:
            return "positive"
    for word in ["bad", "hate", "terrible", "awful"]:
        if word in lowered:
            return "negative"
    return "neutral"
```

The Streamlit app

```
# practice/app_streamlit.py   —   run:  streamlit run app_streamlit.py
import streamlit as st

from pipeline import classify

st.title("Sentiment Classifier")

text = st.text_input("Type something:")
if st.button("Classify"):
    if text.strip():
        st.success(f"Result: {classify(text)}")
    else:
        st.warning("Please type something first.")
```

Run it with `streamlit run app_streamlit.py` and a browser tab opens. The whole script re-runs on every click — that's Streamlit's model.

The chat version (a front for an agent)

A chatbot UI needs memory across re-runs, which is what `st.session_state` is for:

```
# practice/app_chat.py   —   run:  streamlit run app_chat.py
import streamlit as st

from pipeline import classify

st.title("Sentiment Chat")

if "messages" not in st.session_state:
    st.session_state["messages"] = []

for message in st.session_state["messages"]:
    with st.chat_message(message["role"]):
        st.write(message["content"])

user_text = st.chat_input("Say something")
if user_text:
    st.session_state["messages"].append({"role": "user", "content": user_text})
    reply = classify(user_text)          # a real app calls your agent here
    st.session_state["messages"].append({"role": "assistant", "content": reply})
    st.rerun()
```

The chat state, as plain logic (what `st.session_state` holds — testable without the server):

```
def chat_step(session, user_text):
    session.setdefault("messages", [])
    session["messages"].append({"role": "user", "content": user_text})
    reply = classify(user_text)
    session["messages"].append({"role": "assistant", "content": reply})
    return session
```

Expected behavior: the first app shows a box and a button; typing "I love it" and clicking gives "positive". The chat app keeps the conversation on screen — each turn appends a user message and an assistant reply to `st.session_state["messages"]`, so the history survives Streamlit's re-run.

Why this is the fastest UI: no HTML, no routes, no server code — just widgets as function calls and a script that re-runs. For a demo or an internal tool, you are done in minutes. The cost (less

layout control, no real routing) is exactly what Flask and the others buy back.